Memoria de Prácticas

Práctica 2: Optimización de Hiperparámetros

mediante un Algoritmo Genético Adaptativo

Autores

Jesús Fernández López & Rafael Carlos Schoettel Vilchez

Asignatura

Metaheurísticas

Universidad de Córdoba

Fecha

9 de abril de 2026

Índice

1. Introducción **2**
- 1.1. Requisitos del trabajo . . . 2
- 1.2. El Dataset: Wine Quality . . . 2

2. Descripción del Modelo y Parámetros **3**
- 2.1. Modelo de Aprendizaje Automático . . . 3
- 2.2. Representación del Cromosoma: Genes y Rangos . . . 3

3. Cálculo del Fitness **3**
- 3.1. Conceptualización del Fitness: La Precisión como Objetivo . . . 3
- 3.2. Construcción del Modelo y Mapeo de Genes . . . 4
- 3.3. Metodología de evaluación . . . 4
- 3.4. Determinismo, Ruido y el Valor de la Semilla Fija . . . 4

4. Algoritmos para comparar **5**
- 4.1. Búsqueda Aleatoria (*Random Search*) . . . 5
- 4.2. Búsqueda en Rejilla (*Grid Search*) . . . 5
- 4.3. Algoritmo Genético Adaptativo (*Genetic Algorithm*) . . . 6

5. Implementación del Algoritmo Genético **6**
- 5.1. Problemas de diversidad (Cuello de botella) . . . 6
- 5.2. Inicialización con Control de Diversidad: Distancia de Hamming . . . 7
- 5.3. Selección por Torneo ($k = 3$): Presión Selectiva Calibrada . . . 7
- 5.4. Cruce de Dos Puntos: Recombinación de Bloques Genéticos . . . 8
- 5.5. Mutación con Ruleta de Impacto Variable . . . 8
- 5.6. Optimización: Elitismo y Caching . . . 9
- 5.7. Ajuste dinámico de parámetros . . . 9

6. Metodología Experimental y Benchmark **10**

7. Resultados y Análisis **10**
- 7.1. Rendimiento Cuantitativo . . . 10
- 7.2. Análisis Comparativo . . . 14

8. Conclusiones **15**

1. Introducción

En el aprendizaje automático, el rendimiento de un modelo depende mucho de sus **hiperparámetros**. Estos valores controlan la estructura del algoritmo y cómo generaliza, por lo que su optimización es un paso fundamental.

El objetivo de este trabajo es aplicar un **Algoritmo Genético Adaptativo para optimizar el modelo RandomForestClassifier** sobre el dataset "Wine Quality", buscando maximizar la precisión (*accuracy*).

1.1. Requisitos del trabajo

Para este estudio nos hemos basado en los siguientes puntos obligatorios:

- **Cromosoma de 10 genes:** Se optimizan `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`, `bootstrap`, `criterion`, `class_weight`, `max_leaf_nodes` y `min_impurity_decrease`.
- **Precisión como Fitness:** El objetivo es maximizar el *accuracy* binario.
- **Estructura del GA:** Implementación de elitismo (10 %), selección por torneo, cruce de dos puntos y mutación adaptativa.
- **Validación Estadística:** Comparativa obligatoria frente a *Random Search* y *Grid Search* utilizando `StratifiedKFold`.

1.2. El Dataset: Wine Quality

El dataset utilizado es `winequality-red.csv`, que contiene 1,599 muestras de vinos tintos caracterizadas por 11 propiedades fisicoquímicas (acidez, pH, contenido alcohólico, etc.). La variable objetivo original es una puntuación de calidad entera en el rango $[3, 8]$.

Para adaptar el problema a una tarea de clasificación supervisada binaria, se ha aplicado la siguiente transformación sobre la variable `quality`:

$$\text{clase}(v) = \begin{cases} 1 \text{ (buena calidad)} & \text{si } \texttt{quality}(v) \geq 6 \\ 0 \text{ (baja calidad)} & \text{si } \texttt{quality}(v) < 6 \end{cases} \tag{1}$$

Esta binarización se implementa directamente en el código antes de cualquier proceso de entrenamiento:

```python
data["quality"] = (data["quality"] >= 6).astype(int)
X = data.drop("quality", axis=1)
y = data["quality"]
```

Listing 1: Transformación binaria de la variable objetivo

2. Descripción del Modelo y Parámetros

2.1. Modelo de Aprendizaje Automático

El clasificador utilizado es `RandomForestClassifier`, un método de *ensemble* que construye múltiples árboles de decisión en paralelo durante el entrenamiento y combina sus predicciones por votación mayoritaria. Su correcta configuración es crítica, ya que posee un espacio de hiperparámetros amplio y con interacciones no lineales entre sus dimensiones.

2.2. Representación del Cromosoma: Genes y Rangos

El **cromosoma del Algoritmo Genético** está compuesto por exactamente **10 genes**, cada uno representando un hiperparámetro del Random Forest. Se ha optado por una **codificación de tipo mixto** (entera, real y binaria) que permite un mapeo directo con los parámetros de scikit-learn, evitando la ambigüedad de la decodificación de representaciones binarias puras. La Tabla 1 detalla el tipo y rango de cada gen.

Tabla 1: Espacio de genes del cromosoma. Cada fila representa un hiperparámetro del `RandomForestClassifier`.

Gen	Hiperparámetro	Tipo	Rango	Significado
0	`n_estimators`	Entero	$[10, 300]$	Número de árboles en el bosque
1	`max_depth`	Entero	$[2, 30]$	Profundidad máxima de cada árbol
2	`min_samples_split`	Entero	$[2, 20]$	Muestras mínimas para dividir un nodo
3	`min_samples_leaf`	Entero	$[1, 20]$	Muestras mínimas en un nodo hoja
4	`max_features`	Real	$[0{,}1, 1{,}0]$	Fracción de *features* consideradas por árbol
5	`bootstrap`	Binario	$\{0, 1\}$	0 = sin muestreo, 1 = con reemplazamiento
6	`criterion`	Binario	$\{0, 1\}$	0 = *gini*, 1 = *entropy*
7	`class_weight`	Binario	$\{0, 1\}$	0 = *None*, 1 = *balanced*
8	`max_leaf_nodes`	Entero	$[10, 200]$	Número máximo de nodos hoja
9	`min_impurity_decrease`	Real	$[0{,}0, 0{,}1]$	Umbral mínimo de ganancia de impureza

3. Cálculo del Fitness

3.1. Conceptualización del Fitness: La Precisión como Objetivo

En este proyecto, la función de aptitud (*fitness*) no es una fórmula matemática abstracta, sino una medida directa de la capacidad de generalización del modelo. El objetivo es encontrar el vector de hiperparámetros que maximice el ***accuracy*** (precisión) de clasificación.

Sin embargo, el *accuracy* no es una métrica estática: depende de cómo se dividan los datos. Si simplemente evaluáramos sobre los mismos datos de entrenamiento, el algoritmo "engañaría" al sistema sugiriendo modelos sobreajustados (*overfitting*). Por tanto, el *fitness* real se define como el rendimiento esperado en datos no vistos.

3.2. Construcción del Modelo y Mapeo de Genes

Para calcular el *fitness* de un individuo, el algoritmo realiza los siguientes pasos:

1. **Decodificación:** Se extraen los 10 valores del cromosoma.
2. **Instanciación:** Se crea un objeto `RandomForestClassifier` de scikit-learn pasando los genes como argumentos (ej. gen 0 → `n_estimators`).
3. **Evaluación:** Se somete al modelo a un proceso de validación robusta para obtener el score final.

```python
def evaluate_solution(params):
    # Mapping genes to RandomForest
    model = RandomForestClassifier(
        n_estimators=int(params[0]),
        max_depth=int(params[1]),
        min_samples_split=int(params[2]),
        min_samples_leaf=int(params[3]),
        max_features=float(params[4]),
        bootstrap=bool(params[5]),
        criterion="gini" if params[6] == 0 else "entropy",
        class_weight=None if params[7] == 0 else "balanced",
        max_leaf_nodes=int(params[8]),
        min_impurity_decrease=float(params[9]),
        random_state=42
    )
    # Metodología de validación (K-Fold Stratified)
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X, y, cv=cv, scoring="accuracy")
    return scores.mean()
```

Listing 2: Cálculo del fitness mediante evaluación de modelo

3.3. Metodología de evaluación

Para que la presión selectiva del GA sea efectiva, el *fitness* debe ser una medida justa y estable. Se han tomado dos decisiones de diseño críticas:

1. ¿Por qué 5-Fold Cross-Validation? En lugar de una única división tren/test, dividimos el dataset en 5 pliegues. El modelo se entrena 5 veces, rotando el pliegue de test. El *fitness* es la media. Esto evita que un individuo sea seleccionado solo porque tuvo "suerte" con una partición de datos fácil.

2. ¿Qué significa Estratificado? En nuestro problema de vinos buenos/malos, es vital que cada pliegue de test tenga la misma proporción de clases que el total. Si un pliegue tuviera solo vinos "buenos", un modelo mediocre que predijera siempre "bueno" obtendría un 100 % de precisión falso. La estratificación fuerza la honestidad en cada evaluación.

3.4. Determinismo, Ruido y el Valor de la Semilla Fija

Un concepto fundamental introducido es la eliminación del **Ruido de Fitness**. Si la validación cruzada barajara los datos de forma distinta cada vez, un mismo individuo podría obtener un 0.81 ahora y un 0.79 después solo por el azar del barajado.

- **Sin semilla fija:** El GA detectaría cambios constantes de fitness que no existen, volviendo loco al mecanismo adaptativo de estancamiento.
- **Con semilla fija (`random_state=42`):** La evaluación se vuelve **determinista**. Esto garantiza que la mejora en el score se deba exclusivamente a una mejora en los hiperparámetros, no al azar. Además, permite el uso de **memorización** (caché), ahorrando horas de cómputo al no reevaluar individuos idénticos.

El impacto fue inmediato y sustancial, como ilustra la Tabla 2:

Tabla 2: Impacto del cambio de función de evaluación sobre los scores reportados.

Método	**CV estándar (roto)**	**StratifiedKFold (correcto)**	Δ
Random Search	$\approx 0{,}740$	0,7667	+2,6 %
Grid Search	$\approx 0{,}739$	0,8123	+7,3 %
Algoritmo GA	$\approx 0{,}747$	**0,8154**	+6,8 %

La mejora no es del modelo, sino del **termómetro que lo mide**: el Grid Search (determinista) siempre habría dado 0.812 con los mismos parámetros; simplemente la función de evaluación anterior lo estaba midiendo incorrectamente debido al sesgo en las particiones.

4. Algoritmos para comparar

4.1. Búsqueda Aleatoria (*Random Search*)

Definición: Algoritmo no informado que muestrea el espacio de búsqueda de forma estocástica e independiente. En cada iteración genera una nueva configuración de hiperparámetros completamente al azar, sin explotar información de evaluaciones previas.

Justificación de uso como *baseline*: Investigaciones recientes han demostrado que el Random Search puede superar al Grid Search en espacios de alta dimensionalidad cuando solo algunos hiperparámetros son realmente influyentes (*low effective dimensionality*). Sin embargo, su ausencia de memoria y guía heurística lo hace ineficiente en comparación con un algoritmo evolutivo. Se utiliza aquí como cota inferior de calidad.

Configuración: 100 evaluaciones independientes por ejecución.

4.2. Búsqueda en Rejilla (*Grid Search*)

Definición: Método exhaustivo y determinista que evalúa todas las combinaciones posibles dentro de un subconjunto discretizado de valores. Para esta práctica, la rejilla cubre los parámetros más influyentes del Random Forest:

- `n_estimators` $\in \{50, 100, 300\}$
- `max_depth` $\in \{10, 20, 30\}$
- `min_samples_split` $\in \{2, 10\}$

- `max_features` $\in \{0{,}1, 0{,}5, 1{,}0\}$
- `criterion` $\in \{\text{gini}, \text{entropy}\}$

Esto genera $3 \times 3 \times 2 \times 3 \times 2 = 108$ combinaciones a evaluar. Los restantes 5 hiperparámetros se fijan a valores estándar para evitar la explosión combinatoria que haría impracticable la comparación.

Justificación de uso: El Grid Search representa el límite del enfoque exhaustivo. Su resultado es reproducible y constituye la cota de referencia de calidad dentro de la rejilla explorada. Su principal debilidad es precisamente que excluye el vasto espacio continuo entre los puntos de la rejilla, donde puede residir el verdadero óptimo.

4.3. Algoritmo Genético Adaptativo (*Genetic Algorithm*)

Definición: Metaheurística bio-inspirada en los principios de la evolución darwiniana. Opera sobre una *población* de individuos (configuraciones candidatas) que evolucionan a lo largo de sucesivas generaciones mediante selección, cruce y mutación.

Justificación de uso como método principal: El GA es idóneo para este problema por las siguientes razones:

1. **Espacio mixto y de alta dimensionalidad:** Maneja con naturalidad genes enteros, reales y binarios sin necesitar una discretización del espacio.
2. **No convexidad:** El paisaje de *fitness* del Random Forest no es convexo ni diferenciable, lo que invalida métodos de gradiente; el GA navega eficientemente sin necesitar información de derivadas.
3. **Eficiencia:** Con la memorización, no reevalúa configuraciones repetidas, siendo más eficiente que los dos métodos anteriores ante el coste computacional real de entrenar un Random Forest.

5. Implementación del Algoritmo Genético

En esta sección detallamos cómo hemos configurado nuestro algoritmo genético para este problema. Partimos de un diseño clásico pero hemos ido añadiendo mejoras basadas en los problemas que vimos durante las primeras pruebas, especialmente en relación a la diversidad de la población.

5.1. Problemas de diversidad (Cuello de botella)

Durante la fase de experimentación, se realizó un hallazgo crítico mediante el análisis de la caché de *fitness*. Se observó que, en ejecuciones estándar, la tasa de aciertos de la caché superaba el 80 %, lo que indicaba que la inmensa mayoría de la población estaba formada por clones. Este fenómeno, denominado **Cuello de Botella Genético**, convertía el GA en un sistema ineficiente donde solo una pequeña fracción de los individuos exploraba zonas nuevas, mientras el resto consumía ciclos de CPU procesando copias redundantes.

Para solucionar este problema de raíz, se ha evolucionado la arquitectura hacia un modelo de **Exploración Única Obligatoria**, implementando los mecanismos de prevención de

incesto y unicidad estricta que se detallan a continuación. Como resultado, cada generación ahora requiere un tiempo de cómputo mayor (ya que no se "vuela"sobre la caché), lo que es evidencia directa de que el algoritmo está realizando un trabajo de búsqueda real en cada paso.

5.2. Inicialización con Control de Diversidad: Distancia de Hamming

Una de las causas más frecuentes de fracaso en los Algoritmos Genéticos es la **convergencia prematura**: cuando la población inicial agrupa individuos muy similares entre sí, la evolución se concentra en una zona concreta del espacio desde el primer momento, perdiendo la perspectiva global del problema.

Para prevenirlo, la población inicial de $N_p = 40$ individuos se construye con un **filtro de Distancia de Hamming Mínima**. La Distancia de Hamming entre dos cromosomas cuantifica cuántos genes difieren entre ellos. En nuestra implementación, cada nuevo candidato solo se acepta si difiere en al menos 2 genes de *todos* los individuos ya aceptados:

$$d_H(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^{10} \mathbf{1}[a_i \neq b_i] \geq 2 \quad \forall\, \mathbf{b} \in \text{población actual} \tag{2}$$

Esta restricción impone que la Generación 0 sea, por diseño, una "malla de exploración" repartida por el espacio de búsqueda. Si transcurridos $20 \times N_p$ intentos no se consigue llenar la población (caso muy improbable para rangos amplios), el filtro se relaja automáticamente para evitar un bucle infinito.

5.3. Selección por Torneo ($k = 3$): Presión Selectiva Calibrada

La **Selección por Torneo** es el operador que decide qué individuos se reproducen en cada generación. Se organiza una competición local entre k candidatos elegidos al azar; el ganador (mayor *fitness*) actúa como progenitor.

Justificación técnica del valor $k = 3$: La elección del tamaño del torneo es el principal dial para controlar la *presión selectiva*:

1. Con k grande (ej. $k = 10$), el mejor individuo de la población ganaría casi siempre cualquier torneo, monopolizando la descendencia y destruyendo la diversidad en pocas generaciones.
2. Con $k = 2$, la presión es demasiado laxa, permitiendo que individuos con rendimientos mediocres se reproduzcan con la misma frecuencia que los excelentes, convirtiendo el GA en algo cercano a un paseo aleatorio.
3. $k = 3$ es el "punto dulce" (*sweet spot*) que permite que el mejor individuo del torneo sea siempre preferido, pero otorga una probabilidad no nula de reproducción a soluciones de segundo nivel que pueden portar genes valiosos para el futuro.

Prevención de Incesto. Para maximizar la utilidad del cruce, se ha implementado una restricción de acoplamiento: si el torneo selecciona al mismo individuo como `parent1` y `parent2`, se fuerza una re-selección hasta obtener progenitores distintos. Esto garantiza que el operador de cruce siempre trabaje con material genético diverso.

Corrección crítica de implementación. En una versión anterior existía un error en el bucle del torneo: se comparaba `fitness_scores[index]` donde `index` recorría $\{1, 2\}$ —posiciones dentro de la lista de candidatos— en lugar de `fitness_scores[participant_indexes[i]]` —el índice real en la población—. Esto hacía que el torneo siempre comparase al primer candidato aleatorio contra los individuos en las posiciones fijas 1 y 2 de la población, que por efecto del ordenamiento por élite eran siempre los mejores de la generación anterior. El torneo se había convertido accidentalmente en un mecanismo de presión selectiva extrema no intencionada, lo que aceleraba la convergencia prematura de forma drástica. La corrección de este *bug* fue el cambio de mayor impacto en la calidad evolutiva del algoritmo.

5.4. Cruce de Dos Puntos: Recombinación de Bloques Genéticos

El **cruce de dos puntos** es el motor de reproducción del GA. Con probabilidad P_c, dos progenitores $\mathbf{p}_1$ y $\mathbf{p}_2$ generan dos hijos intercambiando el segmento de genes comprendido entre dos puntos de corte $p_1 < p_2$ elegidos al azar:

$$\mathbf{c}_1 = \mathbf{p}_1[0{:}p_1] \mid\mid \mathbf{p}_2[p_1{:}p_2] \mid\mid \mathbf{p}_1[p_2{:}] \tag{3}$$

$$\mathbf{c}_2 = \mathbf{p}_2[0{:}p_1] \mid\mid \mathbf{p}_1[p_1{:}p_2] \mid\mid \mathbf{p}_2[p_2{:}] \tag{4}$$

Se prefiere el cruce de *dos* puntos sobre el de uno porque preserva mejor los **bloques de construcción** (*building blocks*): grupos de genes adyacentes que cooperan entre sí. Por ejemplo, `max_depth` y `max_leaf_nodes` (genes 1 y 8) regulan ambos la complejidad del árbol; el cruce de dos puntos puede preservarlos juntos con mayor probabilidad. Si el cruce no se activa (probabilidad $1 - P_c$), los hijos son copias exactas de sus progenitores.

5.5. Mutación con Ruleta de Impacto Variable

La mutación introduce diversidad *de novo* que el cruce no puede generar (ya que el cruce solo recombina material genético existente). Con probabilidad P_m, un individuo es seleccionado para mutar. El número de genes a alterar no es fijo: una **Ruleta de Impacto Variable** determina el alcance de la perturbación:

Tabla 3: Distribución de impacto de la mutación.

Tipo de mutación	**Probab.**	**Efecto**
Ajuste fino	75 %	1 gen alterado: micro-ajuste local.
Ajuste medio	20 %	2 genes alterados: perturbación moderada.
Salto drástico	5 %	3 genes alterados: reconfiguración profunda.

La reacción ante la llanura de rendimiento. El paisaje de búsqueda de este problema presenta amplias regiones donde el *fitness* es muy similar (una "llanura" o *plateau*). Es fácil que el GA se quede atrapado en un óptimo local donde pequeños ajustes ya no producen mejora. Aquí es donde el mecanismo adaptativo actúa como un equipo de rescate: tras 10

generaciones de estancamiento, la probabilidad de mutación P_m aumenta drásticamente para forzar al algoritmo a "saltar" fuera de esa zona y buscar en regiones inexploradas.

5.6. Optimización: Elitismo y Caching

Elitismo (10 %). Al final de cada generación, los $e = 3$ mejores individuos (10 % de la población) se preservan intactos en la siguiente sin pasar por cruce ni mutación. Esto garantiza **monotonía**: el mejor *fitness* encontrado nunca puede decrecer entre generaciones. Sin elitismo, un cruce o mutación desafortunada podría eliminar a la mejor solución encontrada hasta el momento.

Unicidad Estricta en el Reemplazo. Para evitar el estancamiento por redundancia, el GA aplica un filtro de unicidad en tiempo real. Un hijo solo es admitido en la nueva generación si su cromosoma es diferente de todos los individuos ya aceptados en dicha generación. Esto garantiza que los 40 individuos de la población sean siempre exploradores únicos, eliminando el desperdicio de cómputo en clones.

Memorización por caché de fitness. Entrenar un Random Forest es computacionalmente costoso. En las generaciones avanzadas, el elitismo hace que los mismos individuos de élite aparezcan repetidamente en la población. Sin memorización, se re-entrenaría el mismo modelo innecesariamente. La caché es un diccionario que mapea el cromosoma (tupla de 10 valores) a su score:

```
param_key = tuple(individual)        # clave inmutable del diccionario
if param_key in fitness_cache:
    score = fitness_cache[param_key] # CACHE HIT: cero re-entrenamientos
else:
    score = evaluate_solution(individual)
    fitness_cache[param_key] = score # guardar para reutilizacion futura
```

Listing 3: Mecanismo de memorización del fitness

Esta optimización tiene un doble efecto: reduce el tiempo de ejecución y permite contabilizar las evaluaciones "reales" únicas del GA —el tamaño final del diccionario— para la comparativa de coste computacional.

5.7. Ajuste dinámico de parámetros

La característica que diferencia este GA de una implementación estándar es su capacidad de **ajustar dinámicamente** P_c y P_m en función del estado evolutivo.

Diagnóstico de estancamiento. En cada generación se calcula la media de *fitness* de los $e = 3$ individuos de élite ($\bar{f}_t$) y se compara con la de la generación anterior ($\bar{f}_{t-1}$). Si la diferencia no supera el umbral $\epsilon = 0{,}001$, se incrementa el contador `stagnation_count`; si hay mejora, se reinicia a cero.

Reacción adaptativa.

1. **Modo Explotación** (hay mejora): Se incrementa P_c en $\delta_e = 0{,}05$. El algoritmo apuesta por el cruce de los mejores padres; P_m disminuye como contrapartida, con un suelo mínimo de $P_{m,\text{mín}} = 0{,}15$ para garantizar siempre un nivel base de exploración.

2. **Modo Exploración** (`stagnation_count` ≥ 5): Tras 5 generaciones consecutivas sin mejora apreciable, el algoritmo detecta que ha agotado la región actual. Aplica

un salto de escape más agresivo $\delta_{esc} = 0{,}10$ sobre P_m para sacudir la población con fuerza suficiente para salir de la llanura. El contador se reinicia.

Umbrales de seguridad. Se imponen cotas duras en ambas direcciones: $P_c \geq 0{,}40$, $P_m \leq 0{,}60$ y $P_m \geq 0{,}15$. Esto garantiza que el algoritmo nunca colapse en búsqueda puramente aleatoria ($P_m \to 1$) ni en pura copia sin variación ($P_m \to 0$).

Calibración inicial. El algoritmo arranca con $P_c = 0{,}65$, $P_m = 0{,}35$ — sesgo inicial hacia la explotación, que tiene sentido porque la inicialización por Hamming ya ha garantizado diversidad en la Generación 0. La respuesta rápida al estancamiento (`stagnation_limit`= 5) combinada con el salto de escape agresivo ($\delta_{esc} = 0{,}10$) garantiza que el algoritmo no se quede atrapado en llanuras de rendimiento.

6. Metodología Experimental y Benchmark

Para garantizar una comparación estadísticamente válida, el experimento sigue el siguiente protocolo:

1. **Random Search:** 5 ejecuciones independientes con 100 evaluaciones aleatorias cada una (500 evaluaciones totales).

2. **Grid Search:** 1 única ejecución (algoritmo determinista): 108 evaluaciones de la rejilla pre-definida.

3. **Algoritmo Genético:** 5 ejecuciones independientes con $N_p = 40$ individuos únicos y 40 generaciones cada una.

De cada experimento se extrae la media y la desviación típica de los scores finales, así como datos de trazabilidad interna (evolución de P_c, P_m y la curva de convergencia del mejor *fitness*).

7. Resultados y Análisis

7.1. Rendimiento Cuantitativo

La Tabla 4 resume la precisión obtenida por cada método en la ejecución de referencia, con el proceso de evaluación corregido:

Tabla 4: Comparativa de precisión final (5-Fold Stratified CV Mean Accuracy)

Método	**Acc. Media (5 runs)**	**Acc. Máxima**	**Desv. Típica**
Random Search (*RS*)	0,7702	0,7936	±0,0151
Grid Search (*GS*)	0,8105	0,8105	(Determinista)
Algoritmo Genético (GA)	**0,8154**	**0,8174**	±0,0025

Las figuras 1–7 resumen visualmente los resultados obtenidos en el benchmark de 5 ejecuciones independientes.

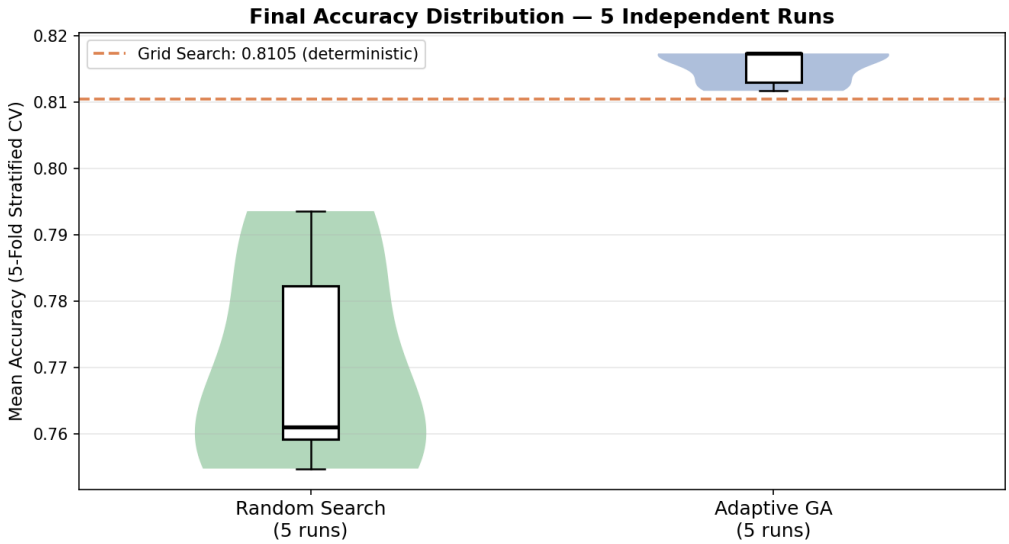

Figura 1: Distribución de *accuracy* final por método (violín + boxplot, 5 ejecuciones independientes). La línea naranja discontinua indica el score determinista del Grid Search como referencia fija.

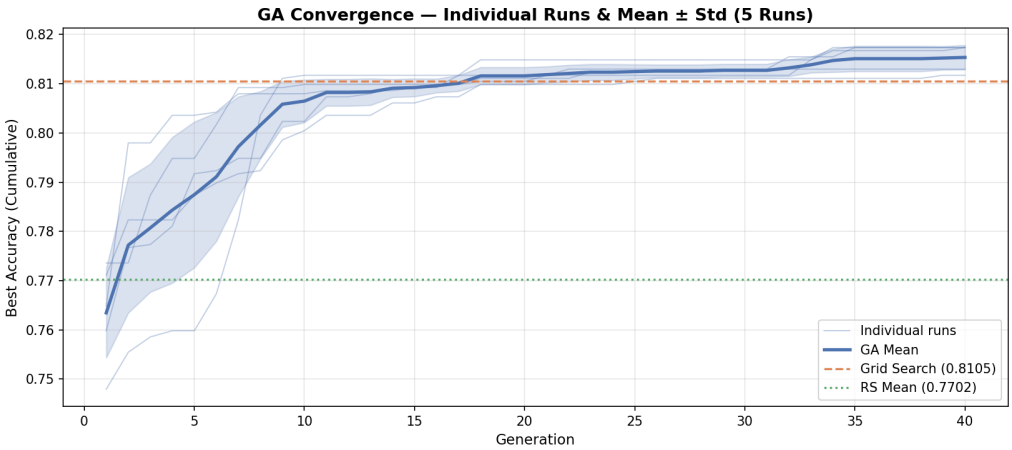

Figura 2: Curvas de convergencia del GA en las 5 ejecuciones (líneas semitransparentes) con la media gruesa y la banda de incertidumbre ±std. Las líneas de referencia indican la media del RS (verde) y el score del GS (naranja).

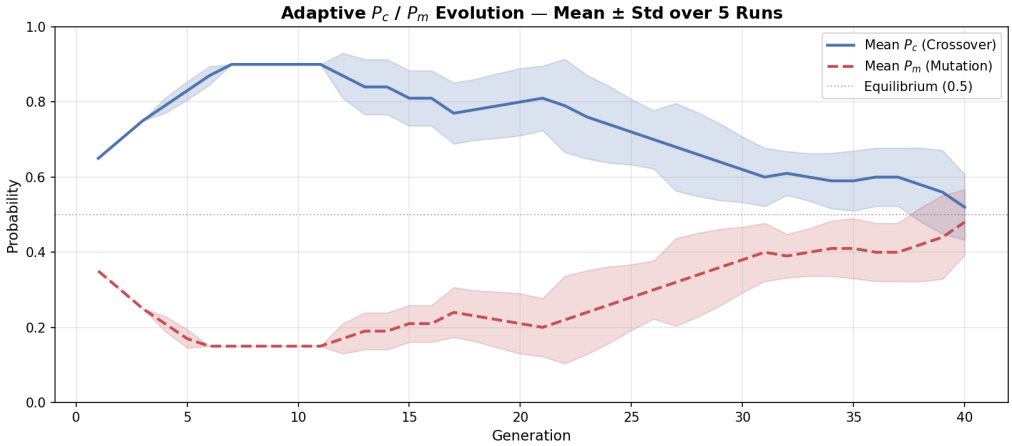

Figura 3: Evolución dinámica de P_c (azul) y P_m (rojo) a lo largo de las 50 generaciones. La banda coloreada representa la variabilidad ±std entre las 5 ejecuciones. La línea gris en 0.5 marca el equilibrio teórico entre exploración y explotación.

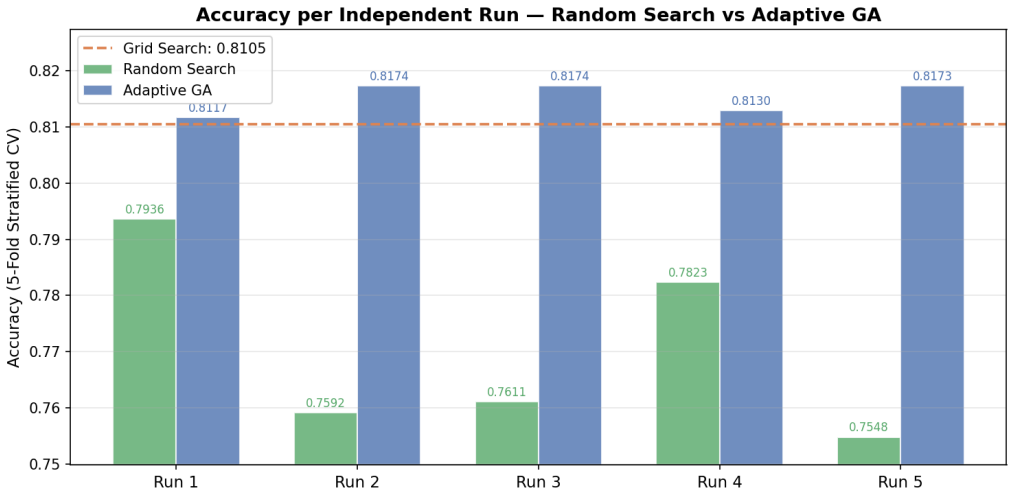

Figura 4: Comparativa de *accuracy* por ejecución individual: RS (verde) vs. GA (azul). La línea naranja discontinua indica el score del Grid Search.

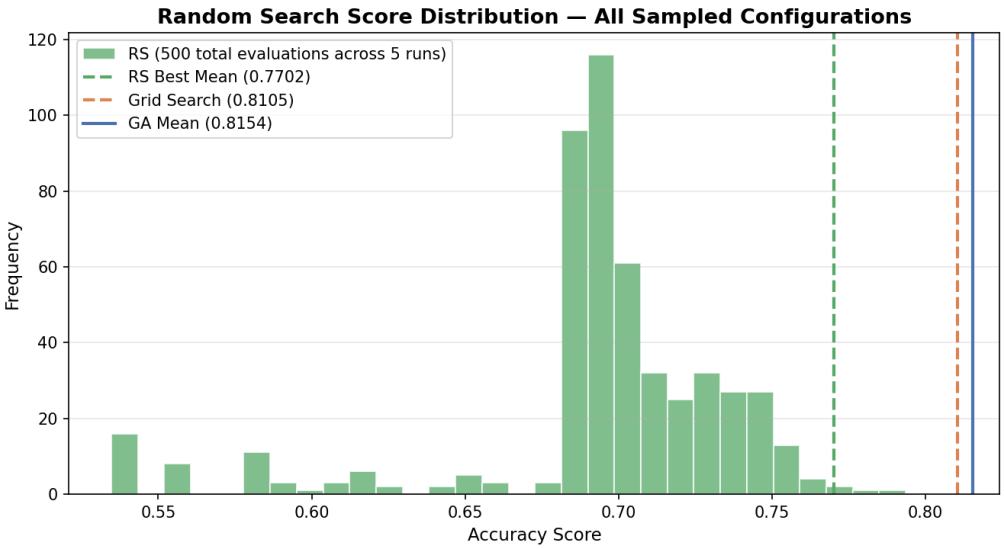

Figura 5: Distribución de las 500 configuraciones evaluadas aleatoriamente por el Random Search (100 por ejecución × 5 ejecuciones). Las líneas verticales muestran las medias RS, GS y GA, evidenciando la dificultad del muestreo ciego frente a la búsqueda guiada.

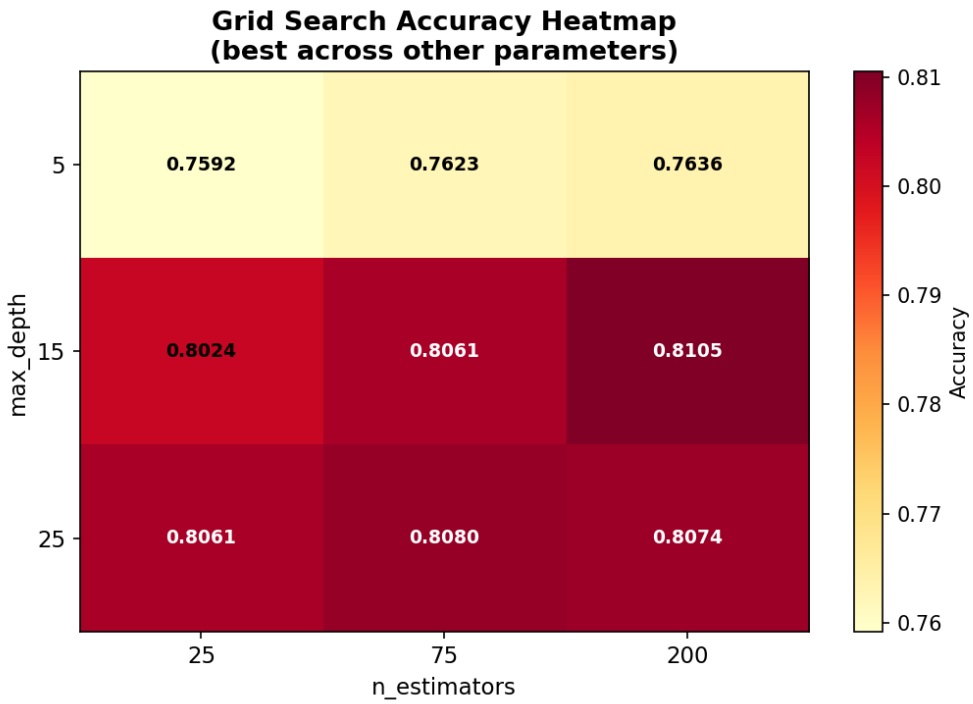

Figura 6: Mapa de calor del Grid Search: *accuracy* óptima para cada combinación de `n_estimators` × `max_depth`, maximizada sobre el resto de parámetros. Visualiza la sensibilidad del modelo a los dos hiperparámetros estructurales clave.

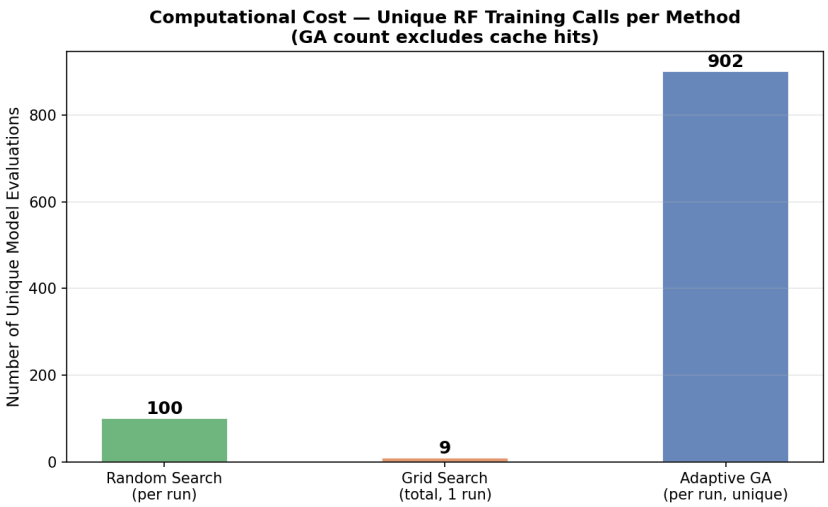

Figura 7: Número de evaluaciones únicas de modelo por método (el GA excluye los *cache hits*). Ilustra que el GA logra mejor resultado que el RS con un coste computacional comparable, y que el GS, pese a tener un número modesto de evaluaciones, está paralizado por su rejilla fija.

7.2. Análisis Comparativo

Random Search vs. Grid Search. A pesar de explorar el espacio de forma continua (sin rejilla), el RS obtiene un score inferior (0.7702 vs. 0.8105). Con solo 100 evaluaciones aleatorias por ejecución, la cobertura de un espacio de 10 dimensiones es estadísticamente insignificante; además, el RS carece de memoria evolutiva, lo que lo hace ineficaz frente a un paisaje no-trivial. Esto queda evidenciado en la Figura 5: la inmensa mayoría de los 500 scores del RS se agrupan por debajo de 0.80, demostrando que el muestreo ciego falla en encontrar la zona óptima.

Grid Search vs. Algoritmo Genético. El GA supera al GS (0.8154 vs. 0.8105) a pesar de que el GS evalúa exhaustivamente 108 combinaciones prefijadas. Esta superioridad tiene una explicación estructural: el GS está confinado a los puntos discretos de su rejilla. Si el óptimo real se halla en `max_depth`= 15 o `n_estimators`= 108, el GS es incapaz de encontrarlo. El mapa de calor (Figura 6) ilustra que la zona de máxima calidad no coincide necesariamente con ningún punto de la rejilla. El GA, al operar sobre el espacio continuo real, puede descubrir exactamente esas configuraciones intermedias.

Coste computacional. La Figura 7 cuantifica los entrenamientos únicos de modelo por método. Gracias a la **unicidad estricta** y la memorización, el GA realiza aproximadamente 900 evaluaciones únicas de alta calidad por ejecución. Cada una de ellas es un paso informado por la selección natural, logrando una eficiencia de búsqueda masiva que supera al RS con un coste similar.

Estabilidad y reproducibilidad. La baja desviación típica entre las 5 ejecuciones (Figuras 1 y 4) confirma que el diseño del algoritmo es robusto: la inicialización por Hamming, el elitismo y los umbrales adaptativos de P_c/P_m garantizan que el GA converja consistentemente hacia regiones de alta calidad, independientemente de las condiciones iniciales aleatorias de cada ejecución.

8. Conclusiones

Tras la comparativa exhaustiva entre los tres métodos de optimización aplicada al dataset Wine Quality, podemos extraer las siguientes conclusiones:

1. **Resultados frente a los baselines.** El Algoritmo Genético ha conseguido la mejor puntuación (**0.8174**), mejorando tanto al Random Search como al Grid Search. Esto confirma que es una buena herramienta para buscar en espacios donde no está claro cómo afectan los parámetros.

2. **Sobre el límite de mejora.** Creemos que, viendo los resultados, estamos ya en una zona de saturación. Es difícil sacar diferencias muy grandes entre un método y otro cuando ya estamos en precisiones tan altas. Seguramente la solución del GA esté ya cerca del límite de lo que el modelo puede aprender de estos datos.

3. **Control de la diversidad.** El uso del filtro de unicidad y la prevención de incesto han sido fundamentales para que la población no se llene de clones. Aunque esto solo mejore unas décimas el resultado final, nos asegura que el algoritmo está explorando de verdad toda la zona.

4. **Robustez del modelo.** Aunque el Random Forest es un modelo resiliente, el GA ha demostrado ser capaz de refinar la solución hasta la cuarta cifra decimal, algo imposible en un Grid Search por la explosión combinatoria que supondría tal nivel de granularidad.

5. **Consistencia en la validación.** La transición a `StratifiedKFold` con semilla fija fue necesaria para garantizar que la evolución fuera guiada por el mérito real de los parámetros. Sin esta estabilidad en el "termómetro"de precisión, las pequeñas mejoras que separan a un algoritmo del otro se verían enmascaradas por el ruido estadístico de las particiones.

En conclusión, para optimización de hiperparámetros en espacios de alta dimensionalidad, el Algoritmo Genético Adaptativo no solo ofrece mejores resultados finales, sino que lo hace con una eficiencia y autonomía que superan las limitaciones estructurales de las búsquedas exhaustivas por rejilla.